1️⃣ .shape – Know the Structure

arr = np.arange(12).reshape(3,4)

arr.shape

Output:

(3,4)

Meaning:

- 3 rows
- 4 columns

For 3D:

arr = np.arange(24).reshape(2,3,4)

arr.shape

(2,3,4)

Meaning:

- 2 blocks
- 3 rows
- 4 columns

👉 Always check .shape before reshaping.

2️⃣ .reshape() – Change Shape (No Data Change)

arr = np.arange(12)

arr2 = arr.reshape(3,4)

Rules:

1. Total elements must match.
2. 12 → 3×4 ✓
3. 12 → 2×5 ✗
4. -1 lets NumPy calculate automatically:

arr.reshape(3,-1)

NumPy computes missing dimension.

⚠ Important:

- Returns **view if possible**
- Doesn't modify original unless reassigned

3️⃣ .resize() – Dangerous One ⚠

arr = np.arange(6)

arr.resize(3,2)

Difference from reshape:

reshape	resize
returns new array	modifies original
safe	can overwrite data

If new size bigger → fills with 0:

arr = np.arange(3)

arr.resize(5)

Result:

[0 1 2 0 0]

👉 Use rarely. Prefer reshape.

4️⃣ .flatten() – Always Copy

arr = np.arange(6).reshape(2,3)

flat = arr.flatten()

Result:

[0 1 2 3 4 5]

⚠ Important:

- Always returns **copy**

- Changes do NOT affect original

5️⃣ .ravel() – Flatten but Smart

flat = arr.ravel()

Difference:

flatten	ravel
always copy	view if possible
slower	faster

👉 Prefer ravel() when performance matters.

6️⃣ .T – Transpose

Switch rows and columns.

arr = np.arange(6).reshape(2,3)

arr.T

From:

[[0 1 2]

[3 4 5]]

To:

[[0 3]

[1 4]

[2 5]]

Shape:

(2,3) → (3,2)

⚠ Returns **view**, not copy.

7️⃣ np.swapaxes() – Swap Any Two Axes

Very useful for 3D+ arrays.

Example:

arr = np.arange(24).reshape(2,3,4)

Shape:

(2,3,4)

Swap axis 0 and 1:

np.swapaxes(arr, 0, 1).shape

Result:

(3,2,4)

Meaning:

- rows and blocks swapped

👉 Use when working with images, ML data, deep learning.

🧠 **Practical Understanding (Very Important)**

🔹 **Reshape vs Transpose**

- reshape() → rearranges layout logically
- transpose() → rotates axes

Example:

arr.reshape(4,3) # different grouping

arr.T # actual axis flip

They are NOT the same.

🔹 **Flatten vs Ravel (Interview Favorite)**

If interviewer asks:

- Want safe independent array → flatten()
- Want fast, memory efficient → ravel()

🔥 **Memory Behavior Summary**

Function	View or Copy
reshape	View (if possible)
resize	Modifies original
flatten	Copy
ravel	View (if possible)
.T	View
swapaxes	View

🚀 **Real Practical Use Cases**

1. Machine Learning:
    - Flatten image → (28,28) → (784,)
    - Use .reshape() or .ravel()
2. Deep Learning:
    - Change from (batch, height, width, channel)
    - Use swapaxes() or transpose()
3. Data Cleaning:
    - Convert 2D to 1D → ravel()

🎯 **Most Important Things to Remember**

1. Total elements must match for reshape.
2. flatten = copy, ravel = maybe view.
3. resize modifies original.
4. Transpose changes axis order, not data count.
5. Always check .shape.

🔥 Bonus Thinking Question

If:

z = np.arange(48).reshape(2,4,6)

What will be shape after:

z.reshape(4, -1)

-1 is automatic columns

Think carefully before running it.